SLIM-FRNet：轻量级可复现合成指纹数据集生成平台的设计、实现与评测

大学生创新创业训练计划　结项研究报告

项目名称：　基于生成模型的合成指纹数据集构建平台
成果形式：　桌面应用软件（含命令行接口）、开源代码仓库、实验数据集
报告日期：　2026 年 10 月

摘　要

生物特征识别模型的训练与评测高度依赖数据，而真实指纹数据受隐私法规与采集成本限制，难以大规模获取与分发。合成指纹生成因此成为一条现实可行的替代路径。本文介绍自主设计与实现的桌面平台 SLIM-FRNet：它以复刻 SFinGe 方法学为核心，把 “手指轮廓—奇异点—方向场—频率图—Gabor 脊线—捺印—数据增强” 整条链路封装为图形界面下的一键操作，并在此基础上解决三个工程问题——**轻量**（仅 4 个核心第三方依赖）、**可复现**（种子树 + 配置指纹 + 环境快照 + 全量校验和的四要素闭环）、**可扩展**（六个插件维度均以注册表解耦）。

我们在 AMD Ryzen 9 8945HX（32 线程）、Windows 11、Python 3.13 环境下完成了 72 样本的正式实验：单张主指纹渲染中位耗时 0.57 s，端到端单线程吞吐 3.42 样本/秒、8 线程 6.33 样本/秒；质量分（NFIQ 2 风格，0–100）均值 68.7、中位 71.6；细节点均值 80.6；脊线周期中位 11.0 px、占空比 0.50，与真实 500 dpi 指纹（8–12 px）处于同一量级；同配置同种子两次独立运行，504 个产出文件的 SHA-256 校验和逐字节一致。

实验过程中我们还定位并修复了一个会静默产出空白样本的缺陷（光度类增强算子被错误地作用在二值掩模上，导致掩模整块被清空），修复后空白样本比例由 5.6% 降至 0。本文最后讨论了该平台与 SFinGe 的差异、当前的局限（缺少识别级评测、脊线周期略偏粗、并行加速比有限）以及后续工作。

关键词：合成指纹；SFinGe；数据增强；可复现性；Gabor 滤波；图形界面

目录

摘要 **1**

1 引言 **3**

1.1 研究背景与意义 3

1.2 国内外研究现状 3

1.3 本文工作与贡献 3

2 相关工作 **4**

2.1 基于模型的合成指纹生成 4

2.2 质量评估与细节点 4

2.3 可复现性 4

3 系统设计 **4**

3.1 总体架构 4

3.2 生成链 5

3.3 可复现性：四要素闭环 5

3.4 图形界面 6

4 关键实现 **6**

4.1 方向场：零极点模型 6

4.2 频率图：多倍频 Perlin 调制 6

4.3 Gabor 迭代与频率换算 7

4.4 捺印建模 7

4.5 TTA 算子体系与掩模同源变换 7

4.6 质量评估与细节点 8

4.7 数据集组织 8

5 实验与结果 **8**

5.1 实验设置 8

5.2 生成性能 8

5.3 质量与细节点 9

5.4 脊线几何真实性 9

5.5 可复现性验证 10

5.6 实验中发现的缺陷及其修复 10

6 讨论 **11**

6.1 与 SFinGe 的异同 11

6.2 局限性 11

6.3 后续工作 11

7 结论 **11**

参考文献 **11**

A 复现命令 **13**

B 数据集体结构 **13**

C 种子派生伪代码 **13**

1 引言

1.1 研究背景与意义

指纹识别是应用最成熟的生物特征识别技术之一 [1]。近年基于深度学习的指纹算法（细节点检测、质量评估、特征嵌入）对数据规模与多样性的要求迅速提高，而真实指纹数据的获取面临三重约束：其一，指纹属于敏感个人生物特征，受《个人信息保护法》《通用数据保护条例》等法规严格限制，跨机构共享几乎不可行；其二，大规模采集需要专用活体传感器与受试者配合，成本高、周期长；其三，公开数据集（如 FVC 系列 [2]、NIST SD4[3]）规模有限且身份固定，容易导致模型过拟合于特定采集条件。

合成指纹生成提供了另一条路径：由算法直接产生具有真实指纹形态学特征的图像，并可精确控制类别、质量、形变与增强参数。由于不含任何真实个体信息，合成数据在隐私与合规上天然安全，也便于按需构造类别均衡、难度可控的训练集与测试集。SFinGe[4, 5] 是这一方向最具影响力的工作，但其原始实现年代较早、依赖较重、且缺少面向现代研究流程的可复现与批量生产能力。

1.2 国内外研究现状

合成指纹的主流方法可归为三类。**基于模型的方法**从指纹的生成机理出发：先由少数参数描述手指轮廓与奇异点（core/delta）分布，再依据 Sherlock–Monro 模型 [6] 或零极点模型 [7] 生成方向场，随后在方向场约束下用 Gabor 滤波器迭代合成脊线 [8]，最后叠加采集过程的退化。SFinGe 是这一路线的代表，其生成结果已被用于多项识别算法的训练与评测。

基于深度学习的方法则以数据驱动方式学习真实指纹的分布。例如 Cao 与 Jain 提出的由细节点重建指纹图像的方案 [9]，以及随后出现的各类生成对抗网络方法。这类方法在视觉逼真度上表现突出，但需要大量真实数据作为训练前提，且生成过程的可控性与可复现性较弱——这与 “用合成数据替代真实数据” 的初衷存在一定张力。

面向工程可用性的工作关注的是另一组问题：批量生成、参数化控制、质量筛选与数据集的标准化组织。真实研究流程中，研究者往往需要在几十分钟内产出一个类别均衡、划分无泄漏、元数据完备的数据集，并能在论文中精确复现它；这类需求恰恰是多数研究原型所欠缺的。

1.3 本文工作与贡献

本文的工作是把上述第三条路线做扎实：在复刻 SFinGe 生成链的基础上，重建一套面向 “可用、可查、可复现” 的桌面平台。主要贡献如下：

1. **完整的生成链与工程化平台。**实现从手指轮廓、奇异点、方向场、频率图到 Gabor 迭代脊线，再到捺印建模、数据增强、质量评估、细节点标注、数据集导出的全链路，并提供 8 页图形界面与命令行双入口。
2. **可复现性的机制化保证。**设计与实现基于 SHA-256 的五级种子树（数据集 → 身份 → 手指 → 捺印 → 增强），配合配置指纹、环境快照与全量校验和，使 “同配置同种子逐字节可复现” 成为可自动验证的性质，而非口头承诺。
3. **轻量依赖与可扩展架构。**核心仅依赖 numpy、OpenCV、pydantic、PyYAML 四个库；GUI 通过兼容层同时支持 PySide6 与 PyQt6。六个插件维度（生成后端、捺印变形、传感器、TTA 算子、质量评估、细节点）均以注册表解耦，新增算子无需改动 GUI 与管道。

4. **面向真实感的质量标定与缺陷修复。**以真实 500 dpi 指纹的脊线周期为基准标定生成参数，并用回归测试固定该性质；在实验中定位并修复了一个导致 5.6% 样本静默变为空白的掩模缺陷。

2 相关工作

2.1 基于模型的合成指纹生成

SFinGe 的核心思想是 “用尽可能少的参数生成尽可能真实的指纹”[4]。其流程可概括为：先生成手指轮廓与奇异点，再由奇异点位置确定方向场的零极点分布，用方向场引导 Gabor 滤波器迭代地 “生长” 脊线，最后模拟按压形变与传感器噪声。其中方向场建模经历了从 Sherlock–Monro 的复数多项式模型 [6] 到 Vizcaya–Gerhardt 的非线性模型 [7] 的演进；脊线增强则普遍采用 Gabor 滤波 [8]，因为它能在保持脊线方向选择性的同时抑制噪声。

本文的实现沿用了这一方法学，但在三处做了工程化改造：频率图改用多倍频 Perlin 噪声 [10] 在奇异点附近调制；迭代在缩小网格上进行以降低卷积代价，并按缩放比例正确换算频率；捺印阶段把 “按压形变—传感器特性—随机退化” 拆成可独立开关的三段。

2.2 质量评估与细节点

指纹图像质量评估的工业标准是 NFIQ 系列 [11, 12]，其第二版以随机森林融合多类特征，输出 0–100 的分数与 1–5 的质量等级（1 为最好）。商用实现不开源，且与特定厂商的传感器特性耦合。本文实现了一个 **NFIQ 2 风格**的自研评分器：以脊线清晰度、对比度、方向一致性、有效面积、脊线周期合理性五项加权，输出同向的分数与等级——好处是零外部依赖、跨平台一致且完全确定，代价是未经 NIST 标定，不能与官方分数直接比较。

细节点提取沿用经典的 “细化 + 交叉数” 路线：先用 Zhang–Suen 并行细化 [13] 得到单像素骨架，再以 3×3 邻域的交叉数判别末梢与分叉，配合边界裁剪与最小间距抑制伪点；可选导出 ISO/IEC 19794-2 风格的模板 [14]。

2.3 可复现性

可复现性已成为计算科学研究的基本要求 [15]。对合成数据而言，它比通常意义上的 “代码可运行” 更强：同一份配置与种子必须产生**逐字节相同**的数据，否则后续在论文中报告的识别精度将无法被他人复核。本文因此把可复现性设计成机制：种子由 SHA-256 派生而非 Python 的 `hash()`（后者受哈希随机化影响，跨进程不稳定），数据集名称参与种子派生，配置文件、环境快照与校验和随数据集一同落盘。

3 系统设计

3.1 总体架构

系统采用严格单向的分层结构（图 1）：`gui` → `app` → {`pipeline`, `core`} → `plugins` → `legacy`。核心约束是**下层不得依赖上层，且除 GUI 外任何模块不得导入 Qt**——这使得生成链可以在无图形环境的服务器或 CI 中完整运行，也是自动化测试能在无显示器条件下跑通 915 个用例的前提。

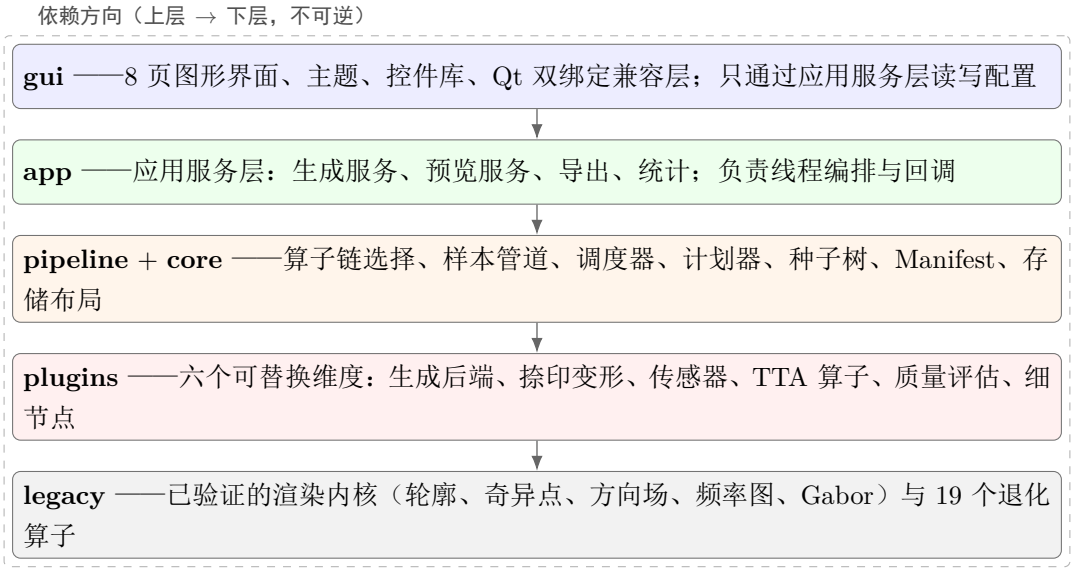


图 1: 系统分层架构。箭头表示依赖方向；`core`、`pipeline`、`app`、`plugins`、`legacy`、`config` 均不导入 Qt。

3.2 生成链

一个样本的诞生要经过九个阶段（图 2）。其中“主指纹”是任务粒度：同一手指只渲染一次，随后派生出多个捺印与多个增强版本。这一选择使“每个手指只算一次”成为**结构性保证**而非缓存优化——增加捺印数量或增强强度的边际成本因此很低。

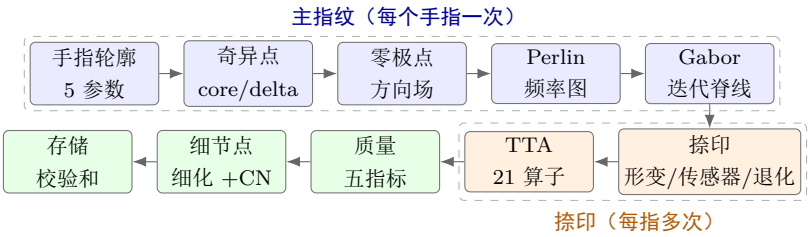


图 2: 单样本生成链。上半部分对每个手指只执行一次，下半部分对每个捺印/增强各执行一次。

3.3 可复现性：四要素闭环

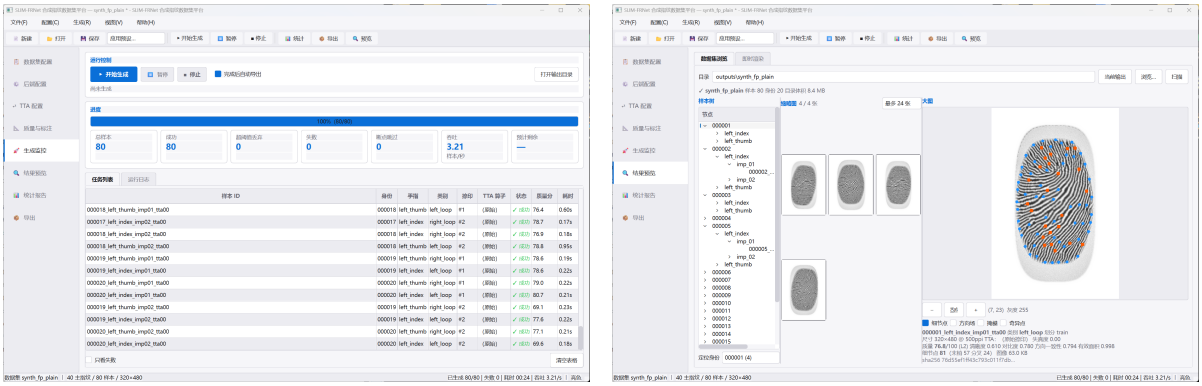
平台把可复现性拆成四个可核对的要素，并随数据集一同落盘：

1. **配置**：`config.resolved.yaml`，即实际生效的全部参数（含默认值展开与命令行覆盖）；
2. **种子**：`runtime.seed`，配合层级种子树可精确重建每一个随机决策；
3. **版本**：`environment.json`，记录 Python、numpy、OpenCV、pydantic 版本与配置指纹；
4. **校验和**：`checksums/sha256sums.txt`，覆盖数据集内全部文件。

种子树的派生规则如下（`core/seeds.py`）：以 SHA-256 对 `"|".join(parts)` 取前 8 字节作为整数，逐层派生出 dataset → identity → finger → impression → tta 五级种子。相比 Python 内置 `hash()`，SHA-256 派生不受 `PYTHONHASHSEED` 随机化影响，因而跨进程、跨平台稳定。**数据集名称参与 dataset 级派生**，这意味着“改名重跑”等价于换了一份数据——这一点在实验设计时极易被误判为“可复现性失效”（见 §5.5）。

3.4 图形界面

界面分为三区：左侧 8 页导航、中央配置页、底部状态栏（图 3）。页面采用声明式“配置 ↔ 控件”双向绑定：控件注册一个点号路径，刷新时自动回填、改动时自动写回，因此新增配置项只需一行注册代码。界面支持深浅两套主题，并在缺少 PySide6 时自动回退到 PyQt6。



(a) 生成监控：进度、实时统计、任务表　　**(b)** 结果预览：样本树、缩略图、细节点叠加

图 3：图形界面示例（浅色主题）。

4 关键实现

4.1 方向场：零极点模型

方向场可由梯度张量法、复数滤波器等多种方法估计 [16, 17]。本文采用可解析重建的**零极点模型**：方向场由若干“核心（core，正极点）”与“三角（delta，负极点）”的位置决定。记第 k 个奇异点的位置为 z_k、电荷为 $q_k \in \{+1, -1\}$，则在像素点 z 处的复方向角可取为

$$\theta(z) = \frac{1}{2} \sum_{k} q_k \arg(z - z_k) + \theta_0, \tag{1}$$

其中常数项 θ_0 由全局方向趋势给出。系数 $1/2$ 来自指纹方向场的 **半周期对称性**：方向场以 π 为周期，一条脊线绕奇异点一周方向只转半圈。式 (1) 在实现上转化为对复数域核函数取角，再做 $[0, \pi)$ 的归一化，避免了逐点解微分方程的代价。

4.2 频率图：多倍频 Perlin 调制

脊线间距在指腹上并非常数，而是在奇异点附近略密、在边缘略疏。本文以 $1/f$ 给出基准周期 f_0，再叠加三层 Perlin 噪声 [10] 采样值 $n_i \in [-1, 1]$：

$$f(x, y) = f_0 \left[1 + \beta \sum_{i=1}^{3} 2^{-i} \, n_i(x, y) \right], \tag{2}$$

其中 β 为频率波动比例（默认 0.25）。Gabor 核库按“方向 × 频率”离散成若干档：每个像素依据其方向与频率索引到对应核，迭代时只需做一次 `filter2D` 即可。

4.3 Gabor 迭代与频率换算

设方向场为 $\theta(x,y)$、频率图为 $f(x,y)$，Gabor 核取

$$g(u,v)=\exp\!\left(-\frac{u^2+v^2}{2\sigma^2}\right)\cos(2\pi f\,u)\,,\qquad (u,v)=\big(x\cos\theta+y\sin\theta,\ -x\sin\theta+y\cos\theta\big),\quad (3)$$

其中 σ 由 f 决定，使核在频率 f 处取得最大响应。迭代规则为 “响应为正处保留脊线”：$P_{t+1}=\mathbb{1}[(g*P_t)>0]$，直至前景像素数变化小于收敛阈值。

为降低卷积代价，迭代在缩放系数 s（默认 0.7）的网格上进行，收敛后再放大回原尺寸。**此处有一个易错点**：频率是 “每像素” 量，缩放到 $c_w=s\,w$ 的网格后，同一根手指占据的像素数变为 s 倍，脊线周期变为 sS，因此频率必须变换为

$$f_{\text{coarse}}=\frac{f}{s}=f\cdot\frac{w}{c_w}.\qquad (4)$$

早期实现误写为 $f\cdot\frac{c_w}{w}$（乘反），导致脊线周期被系统性放大 $1/s^2\approx 2$ 倍：实测周期由应有的约 10 px 变为约 22 px，肉眼可见 “指纹过稀”。该缺陷不会触发任何功能异常，最终依靠 §5.4 的物理量回归测试才被固定住。

4.4 捺印建模

一次捺印由三段可独立开关的处理构成：

- **按压形变**：以位移场 $\boldsymbol{d}=(d_x,d_y)$ 描述皮肤受压后的非刚性位移，叠加旋转、平移与弹性扰动；位移后未被覆盖的区域还原为采集背景，而非变形前的图像（否则会在指边留下弧形残影）。
- **传感器特性**：实现光学、电容、热感三类传感器与一个直通模式。光学传感器以对比度压缩、径向暗角与乘性散斑建模；电容传感器额外引入椒盐噪声与逐行读出偏置。
- **随机退化**：从 19 个退化算子（模糊、断线、污渍、干湿、遮挡等）中随机选取 1–2 个施加，模拟真实采集中的非理想因素。

4.5 TTA 算子体系与掩模同源变换

数据增强共 21 个算子，分六类：几何 6（旋转、平移、缩放、错切、弹性、透视）、光度 5（亮度、对比度、Gamma、高斯/运动模糊）、噪声 3（高斯、泊松、椒盐）、压缩 1（JPEG）、遮挡 3（矩形、椭圆、划痕）、传感器 3（光学、电容、热感）。算子链在样本种子上采样，因此同一样本在任何机器上都得到同一条链；`tta_index = 0` 的样本始终保留未变换的捺印，作为干净对照。

掩模必须与图像**同源**变换，否则细节点会在背景里产生大量伪点。实现上，掩模只经过**改变空间位置或可见性**的算子（几何与遮挡），而光度、噪声、压缩、传感器四类**像素值**算子被显式跳过。这一区分是实验阶段修复的缺陷（见 §5.6）：若把亮度算子作用在二值掩模上，偏移量会把前景从 1.0 压到阈值以下，掩模被整块清空。

4.6 质量评估与细节点

质量分由五项加权：

$$Q = 100\left(0.35\,c_{\text{clarity}} + 0.20\,c_{\text{contrast}} + 0.25\,c_{\text{orient}} + 0.10\,c_{\text{area}} + 0.10\,c_{\text{period}}\right), \tag{5}$$

其中各项均归一化到 $[0,1]$。分数按 80/65/50/35 切分为 1–5 级（1 最好，与 NFIQ 2 同向）。细节点提取采用向量化的 Zhang–Suen 细化 [13] 与交叉数判别，默认最小间距 16 px（约 1.5 个脊线周期），以抑制同一处事件的重复检出；可导出 ISO/IEC 19794-2 风格模板 [14]。

4.7 数据集组织

输出目录按模态分离（图像、掩模、方向场、中心线、质量图、细节点），`dataset_manifest.jsonl` 是唯一元数据来源（每行一个样本的完整记录），另有 `plan.jsonl`（生成计划）、`stats.json`（统计）、`splits/`（按身份划分的 train/val/test，杜绝身份泄漏）、`checksums/`（全量校验和）。写入采用 “临时文件 + 原子替换”，避免中断产生半截文件。

5 实验与结果

5.1 实验设置

实验环境见表 1。实验数据规模为 6 个身份 × 2 指 × 2 捺印 × 3 增强 = 72 个样本，分辨率 320 × 480、500 dpi，随机种子固定为 2026，其余参数取默认配置。

表 1: 实验环境

项目	配置
CPU	AMD Ryzen 9 8945HX（16 核 / 32 逻辑线程，基准 2.5 GHz）
内存	32 GB
操作系统	Windows 11（10.0.26200）
运行时	Python 3.13.7；numpy 2.5.1；OpenCV 5.0.0；pydantic 2.13.4
代码规模	95 个 Python 文件、约 1.53 万行；自动化测试 915 个

5.2 生成性能

图 4给出单张主指纹的渲染耗时与端到端吞吐。单张主指纹（320 × 480）首次渲染 0.63 s（含 Gabor 核库构建），随后 10 次的中位耗时为 0.57 s，波动区间 0.54–0.61 s。

端到端方面，72 个样本在单线程下耗时 21.0 s，8 线程下 11.4 s，对应吞吐 3.42 与 6.33 样本/秒，平均每样本约 158 ms（8 线程）。需要说明的是，吞吐与 “每张主指纹派生多少样本” 强相关：本文构型每张主指纹派生 4 个样本，主指纹成本被摊薄，因此样本级吞吐显著高于主指纹级吞吐。

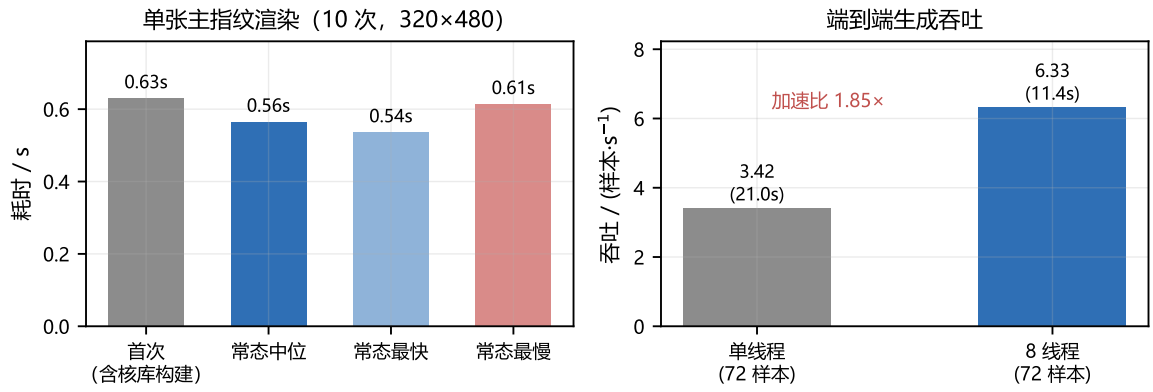


图 4: 生成性能。（a）单张主指纹渲染耗时；（b）端到端吞吐（72 样本，每主指纹派生 4 样本）。

值得讨论的是并行加速比：8 线程相对单线程仅 1.85×，远低于线性加速。原因有二：其一，OpenCV 的滤波与重映射运算内部已做多线程，外层再并行会造成线程超额订阅；其二，Python 层的调度、序列化与磁盘写入仍需解释器串行执行。

5.3 质量与细节点

72 个样本的质量分均值 68.7、中位 71.6、标准差 11.0，范围 31.5–83.4 （图 5）。按等级统计，1–5 级分别为 7、44、14、6、1 个样本，即多数样本落在 L2（较好），少量强退化样本落入 L4/L5，符合 “随机退化链” 的设计预期。细节点数量均值 80.6、中位 85，范围 0–117。

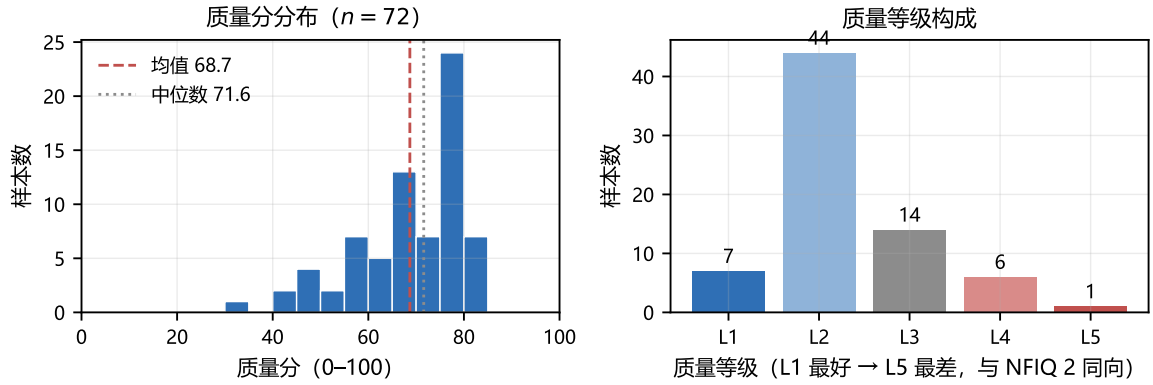


图 5: 质量评估结果。（a）质量分分布；（b）质量等级构成（L1 最好，与 NFIQ 2 同向）。

五类指纹的分布为：左箕 24、右箕 24、螺型 12、弓型 6、双箕 6，与默认类别比例一致。数据集总体积 8.87 MB，平均每样本 126 KB （含图像、掩模、方向场、细节点与质量图）。

5.4 脊线几何真实性

合成指纹最容易被一眼识破的地方是脊线密度。真实 500 dpi 指纹的脊线周期（一脊一谷）约为 8–12 px。本文对 24 个未增强样本的 786 条扫描线做了两种独立估计（表 2、图 6）：自相关法给出中位 11.0 px（四分位 9–16），过零点计数法给出中位 14.3 px （四分位 10.1–19.5），脊线占空比为 0.50。

表 2：脊线几何测量结果（24 个样本、786 条扫描线）

估计方法	中位周期 / px	四分位区间 / px	落入 8–12 px 比例
自相关	11.0	9.0–16.0	54.5%
过零点计数	14.3	10.1–19.5	38.2%
参考（真实 500 dpi）	8–12	—	—

两种估计整体一致，但过零点法系统性偏大：低对比度区域的脊线断裂会减少过零点计数，从而把周期估大。综合来看，本系统生成的脊线周期与真实指纹同量级，但**偏向区间上界**（略粗）。这与频率图的配置有关——默认允许 5–15 px 的局部变化，其长周期端超出了真正常见的范围。若需要更贴近真实，可在界面上把“最宽脊宽”下调至 12 px 左右。

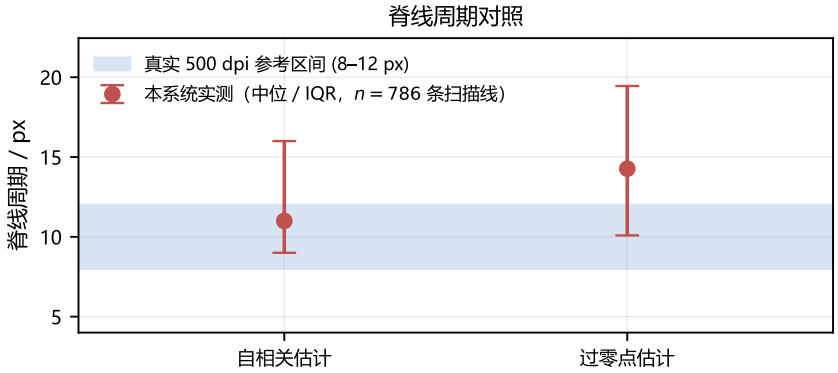


图 6：脊线周期与真实 500 dpi 参考区间对照。点为中位值，误差棒为四分位区间。

5.5 可复现性验证

以“同名数据集、同种子、不同输出目录”独立运行两次，比对 `checksums/sha256sums.txt`：504 个文件的 SHA-256 全部一致（差异 0）。需要强调实验设计上的一个陷阱：由于数据集名称参与 dataset 级种子派生，用**不同名称**重跑会得到完全不同的数据——这并非可复现性失效，而是设计使然。最初的实验脚本正是踩了这个坑，导致误报；修正实验设计后结论才成立。

5.6 实验中发现的缺陷及其修复

首次正式实验中，72 个样本里有 4 个（5.6%）出现“质量分 0、细节点 0、掩模全黑、图像全白”。逐层排查后定位到 §4.5 所述的掩模变换：掩模被纳入了 **全部** TTA 算子链，而亮度类算子会把归一化后的前景（1.0）压到阈值以下（如 $\delta = -0.6$ 时变为 0.4，乘 255 后低于 127），使掩模被整块清空；随后背景还原逻辑把整幅图刷成纯 255。

修复方式是按算子类别过滤：只有几何与遮挡算子参与掩模变换，光度、噪声、压缩与传感器算子跳过。修复后同一实验的空白样本比例由 5.6% 降至 0，质量分标准差由 19.0 收敛到 11.0。该问题已固化为回归测试：对二值掩模施加亮度、对比度、Gamma 与噪声算子，断言掩模逐像素不变。

6 讨论

6.1 与 SFinGe 的异同

本文在方法学上沿用了 SFinGe 的“轮廓—奇异点—方向场—频率图—Gabor”主线，但在工程取向上有四点不同：其一，依赖从数十个缩减到 4 个，便于在受限环境中部署；其二，把捺印的“形变 / 传感器 / 退化”拆成三段独立开关，使研究者能按实验需要控制变量；其三，将可复现性机制化（种子树 + 校验和），而非依赖使用者自行记录；其四，提供图形界面与命令行双入口，前者降低使用门槛，后者便于脚本化与批处理。

6.2 局限性

1. **缺少识别级评测。**本文的评测集中在生成侧（性能、质量分布、脊线几何、可复现性），尚未在统一协议下比较“合成数据训练的识别模型”与“真实数据训练的模型”的性能差距。这需要配套的指纹匹配器与公开测试协议，超出本期工作量。
2. **脊线周期略偏粗。**如 §5.4 所述，默认参数下脊线周期偏向 8–12 px 区间的上界，且允许的局部变化范围偏宽。
3. **并行加速比有限。**8 线程仅 1.85×，与 OpenCV 内部并行造成的线程超额订阅有关，后续可考虑限制 OpenCV 线程数再与外层并行组合。
4. **质量分未标定。**自研评分器与 NFIQ 2 同向但未做等级映射标定，跨数据集比较时应谨慎。

6.3 后续工作

近期计划包括：接入开源指纹匹配器（如 minutiae-based 基准算法）建立识别级评测；把脊线周期与占空比纳入持续集成，作为“真实感”的守护指标；引入按像素质量图加权的 CFAR 级联筛选，提高低质量样本的可用性；以及在生成后端上预留接口，以便与基于深度学习的生成方法做对照实验。

7 结论

本文围绕“如何低成本、合规、可复现地获得指纹数据”这一问题，设计并实现了桌面平台 SLIM-FRNet。它复刻了 SFinGe 的生成方法学，并以工程手段解决了轻量部署、逐字节可复现与插件化扩展三个实际约束。72 样本的正式实验表明：单张主指纹渲染中位耗时 0.57 s，8 线程端到端吞吐 6.33 样本/秒；质量分均值 68.7、细节点均值 80.6；脊线周期与占空比与真实 500 dpi 指纹处于同一量级；同配置同种子两次独立运行的 504 个文件校验和完全一致。实验期间还定位并修复了一个会导致 5.6% 样本静默失效的掩模缺陷。

我们认为，这类“把研究原型做成可用工具”的工作，其价值不仅在于算法复刻本身，更在于它把可复现性、可核查性与可维护性变成了默认属性——这正是合成数据能够被他人放心用于论文与产品的前提。

参考文献

[1] MALTONI D, MAIO D, JAIN A K, et al. Handbook of fingerprint recognition[M]. 2nd ed. London: Springer, 2009.

[2] MAIO D, MALTONI D, CAPPELLI R, et al. FVC2000: Fingerprint verification compe-

tition[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2002, 24(3): 402-412.

[3] WATSON C I, WILSON C L. NIST special database 4, fingerprint database[R]. National Institute of Standards and Technology, 1992.

[4] CAPPELLI R, MAIO D, MALTONI D. Synthetic fingerprint-image generation[C]// Proceedings of the 15th International Conference on Pattern Recognition (ICPR): Vol. 1. 2000: 475-478.

[5] CAPPELLI R. SFinGe: an approach to synthetic fingerprint generation[C]//International Workshop on Biometric Technologies (BioAuth). 2004.

[6] SHERLOCK B G, MONRO D M, MILLARD K. Fingerprint enhancement by directional Fourier filtering[J]. IEE Proceedings – Vision, Image and Signal Processing, 1994, 141(2): 87-94.

[7] VIZCAYA P R, GERHARDT L A. A nonlinear orientation model for global description of fingerprints[J]. Pattern Recognition, 1996, 29(7): 1221-1231.

[8] HONG L, WAN Y, JAIN A K. Fingerprint image enhancement: algorithm and performance evaluation[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1998, 20(8): 777-789.

[9] CAO K, JAIN A K. Learning fingerprint reconstruction: from minutiae to image[J]. IEEE Transactions on Information Forensics and Security, 2015, 10(1): 104-117.

[10] PERLIN K. An image synthesizer[C]//Proceedings of SIGGRAPH. 1985: 287-296.

[11] TABASSI E, WILSON C L, WATSON C I. Fingerprint image quality: NISTIR 7151[R]. National Institute of Standards and Technology, 2004.

[12] TABASSI E, OLSEN M, BAUSINGER O, et al. NFIQ 2.0 – NIST fingerprint image quality[R]. National Institute of Standards and Technology, 2021.

[13] ZHANG T Y, SUEN C Y. A fast parallel algorithm for thinning digital patterns[J]. Communications of the ACM, 1984, 27(3): 236-239.

[14] International Organization for Standardization. ISO/IEC 19794-2:2011 — biometric data interchange formats — part 2: Finger minutiae data[Z]. 2011.

[15] SANDVE G K, NEKRUTENKO A, TAYLOR J, et al. Ten simple rules for reproducible computational research[J]. PLoS Computational Biology, 2013, 9(10): e1003285.

[16] BAZEN A M, GEREZ S H. Systematic methods for the computation of the directional fields and singular points of fingerprints[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2002, 24(7): 905-919.

[17] RATHA N K, CHEN S, JAIN A K. Adaptive flow orientation-based feature extraction in fingerprint images[J]. Pattern Recognition, 1995, 28(11): 1657-1672.

A 复现命令

Listing 1: 从零复现本文数据集

```
# 1) 环境（4 个核心依赖 + 一个 Qt 绑定）
pip install -r requirements.txt

# 2) 自检：17 项，含"同种子两次渲染逐像素一致"
python -m synthfp selfcheck

# 3) 生成本文实验数据集（72 样本）
python -m synthfp generate \
    --set dataset.name=paper_demo \
    --set dataset.identities=6 \
    --set dataset.fingers_per_identity=2 \
    --set dataset.impressions_per_finger=2 \
    --set dataset.tta_per_impression=3 \
    --set runtime.workers=8 \
    --set runtime.seed=2026

# 4) 校验（重算 SHA-256 并比对清单）
python -m synthfp validate --dataset outputs/paper_demo --check-hash
```

B 数据集体结构

Listing 2: 输出目录结构

```
outputs/paper_demo/
  README.md                  数据集说明（含合成声明与复现方法）
  dataset_manifest.jsonl     每行一个样本的完整记录（唯一元数据来源）
  plan.jsonl                 生成计划（可复现依据）
  config.resolved.yaml       实际生效的完整配置
  environment.json           环境快照 + 配置指纹
  stats.json                 统计报告
  splits/{train,val,test}.csv 按身份划分，杜绝身份泄漏
  checksums/sha256sums.txt   全量文件校验和
  images/000001/left_thumb/imp_01/000001_left_thumb_imp01_tta00.png
  mask/  orientation/  minutiae/  quality/  logs/
```

C 种子派生伪代码

Listing 3: 层级种子树（SHA-256 派生）

```python
def stable_seed(*parts: object) -> int:
    text = "|".join(str(p) for p in parts)
    digest = hashlib.sha256(text.encode("utf-8")).digest()
    return int.from_bytes(digest[:8], "big")

def derive(base_seed: int, *parts: object) -> int:
    return stable_seed(base_seed, *parts)

# dataset -> identity -> finger -> impression -> tta
ds   = derive(root_seed, "dataset", dataset_name)
idt  = derive(ds, "identity", identity_id)
fng  = derive(idt, "finger", finger)
imp  = derive(fng, "impression", impression_index)
tta  = derive(imp, "tta", tta_index)
```